VIT®

BHOPAL

www.vitbhopal.ac.in

PROJECT REPORT

Vanisha Srivastava

25BCE11245

B.Tech Computer Science (core)

CSA2001

Introduction

Students often face confusion when planning their academic trajectory due to complex dependency chains between courses. Manual verification of prerequisites is time-consuming and prone to human error, which can lead to administrative issues or academic failure if a student lacks the foundational knowledge (e.g., attempting **Advanced NLP** without **Formal Logic**).

The Goal: To build a rule-based logic engine that automates the verification of course eligibility using core AI and Knowledge Representation concepts.

Relevance to Course Concepts

This project serves as a practical application of several key modules from the course:

- **Knowledge Representation:** I utilized **Structural Knowledge** (mapping objects and their relationships) and **Declarative Knowledge** (storing facts about courses) using Python dictionaries.
- **Production Rules:** The system operates on "If-Then" logic: *If* a student has completed A and B, *then* they are eligible for C.
- **Logical Conjunction (\land):** The core engine implements the logical **AND** operator.
- **Search Algorithms:** The system performs a membership search within the Knowledge Base to validate state.

Technical Approach & Architecture

The project is structured into three distinct layers:

1. **The Knowledge Base (KB):** A centralized dictionary containing course codes, titles, and a list of prerequisites.
2. **The Inference Engine:** A Python function check_eligibility that acts as the "reasoner." It compares the User State (completed courses) against the KB constraints.
3. **The User Interface:** A Command Line Interface (CLI) that gathers user input, processes it through the engine, and outputs a clear success or failure message.

Challenges

Choice of Language: I chose **Python** because of its readability and excellent handling of data structures (lists and sets), which are essential for comparing academic records.

Challenge: Data Normalization: Users often input course codes in different formats (e.g., "cs101" vs "CS 101"). I solved this by implementing string normalization (upper-casing and whitespace removal) to ensure the logic engine remains robust.

Decision on Recursion: While the current version handles direct prerequisites, I designed the logic to be easily expandable into a recursive "Depth-First Search" to check for nested prerequisites in future versions.

Conclusion

This project bridged the gap between abstract logical theory and functional software. By representing academic rules as a computer-readable Knowledge Base, I created a tool that provides immediate value. **What I Learned:** The hardest part of AI is often not the algorithm itself, but how the knowledge is structured. A well-organized Knowledge Base makes the logic engine significantly simpler to write.